Chain Mind - RAG Chatbot Project Report

Student: Bianca Charlotin

Project: RAG Chatbot for LangChain Documentation

Application: Chain Mind

---

1. Chosen Website & Justification: LangChain Official Documentation:

Website: https://docs.langchain.com/oss/python/langchain/

LangChain was chosen as the target website for several reasons:

- **Public availability** - The documentation is fully publicly accessible with no authentication required
- **Ethical scrapability** - Documentation sites are designed to be read and referenced freely
- **Rich technical content** - The docs cover a wide range of topics including agents, models, tools, retrieval, RAG, memory, and more, making it ideal for a Q/A chatbot
- **Relevance** - Since LangChain is the core framework used to build this project, using its own documentation as the knowledge base creates a self-referential and educationally meaningful application
- **Well-structured content** - Each documentation page covers a distinct topic, making chunking and retrieval more effective

Pages Scraped (15 total):

- Overview, Quickstart, Agents, Models, Messages, Tools, Retrieval, RAG, Knowledge Base, Context Engineering, Short-Term Memory, Long-Term Memory, Structured Output, Streaming, Middleware

---

2. Scraping Methodology & Challenges Faced

Methodology

Web scraping was implemented using Python's requests library for HTTP requests and BeautifulSoup4 for HTML parsing. The scraping pipeline followed these steps:

1. Fetch - requests.get(url) sends an HTTP GET request to each documentation page

2. Parse - BeautifulSoup(response.text, "html.parser") parses the raw HTML into a structured object

3. Clean - Navigation bars, footers, scripts, and style tags are removed using .decompose()

4. Extract - .get_text(separator="\n", strip=True) extracts clean readable text

5. Save - Each page is saved as a JSON file ({"url": ..., "content": ...}) in data/raw/

Challenges Faced

- Dynamic content - Some modern documentation sites render content via JavaScript. LangChain's docs were largely static and scrapable with requests, avoiding the need for Selenium
- Noise in scraped text - Initial scraping included navigation menus, cookie notices, and repeated footer content. This was resolved by explicitly removing nav, footer, script, and style tags before text extraction
- Model deprecation - During development, the Groq model llama3-8b-8192 was decommissioned mid-project and had to be replaced with llama-3.3-70b-versatile

---

3. Q/A Generation Strategy

Approach

Rather than auto-generating Q/A pairs using an LLM, a manually curated Q/A dataset was created for the LangChain Overview page. This decision was made to ensure:

- Accuracy - Manually reviewed answers are more reliable than auto-generated ones
- Quality control - Each Q/A pair was verified against the source documentation
- Consistency - All questions follow a consistent format suitable for semantic matching

Dataset Format

The dataset is stored as qa/qa_dataset.csv with three columns:

Sample Q/A Pairs

Why Manual Over Generated?

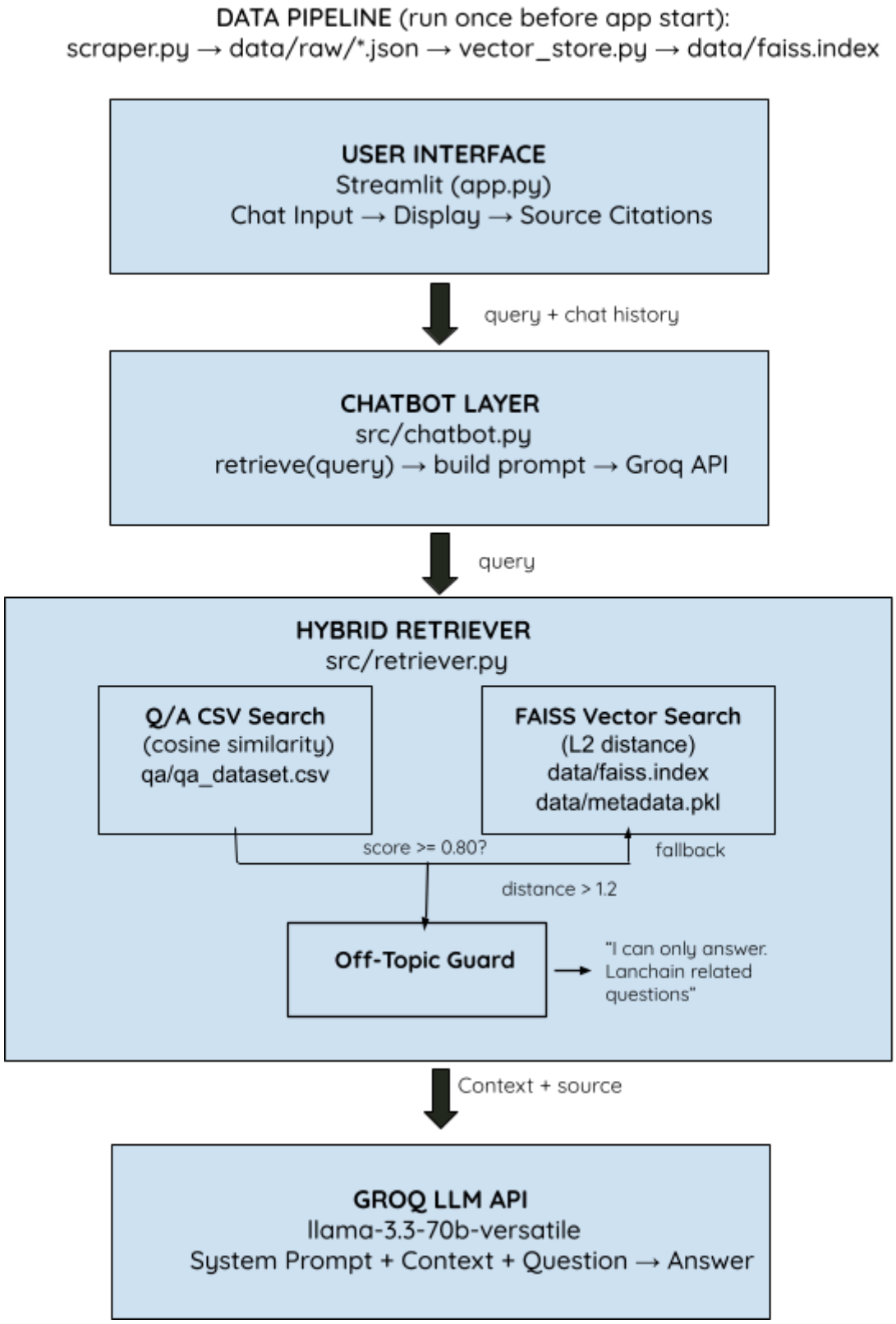
Column	Description
question	A natural language question about LangChain
answer	A concise, accurate answer

source_page	The URL of the documentation page
Question	Answer
What is LangChain?	LangChain is an open-source framework for building LLM-powered agents and applications
Is LangChain open source?	Yes. LangChain is an open-source framework
What is an agent in LangChain?	An agent is an LLM-powered system that can use a model, tools, and instructions to complete tasks

Auto-generation using an LLM prompt like _"Generate 10 Q/A pairs from this content"_ can produce hallucinated or imprecise answers. For a documentation chatbot where accuracy is critical, manually curated pairs provide a higher-quality first layer of retrieval.

---

4. Architecture Diagram



5. Hybrid Retrieval Explanation

Overview

The retrieval system uses a two-stage hybrid approach:

Stage 1 - Q/A Semantic Search

The user's question is converted to a 384-dimensional embedding using the all-MiniLM-L6-v2 model from sentence-transformers. This embedding is compared against all pre-computed question embeddings from the CSV using cosine similarity:

Cosine similarity was implemented manually using NumPy instead of LangChain's built-in util.cos_sim() as an intentional educational decision to better understand the mathematics behind semantic similarity:

- **Dot product** — multiplies matching vector positions and sums the results, measuring directional alignment
- **||v||` (L2 norm)** — represents the Euclidean length of a vector: $\sqrt{(n_1^2 + n_2^2 + \ldots + n_{384}^2)}$
- **Division by both norms** — scales the score between -1 and 1, ensuring similarity reflects semantic direction rather than text length

If the best match scores ≥ 0.80, the pre-written answer is returned directly. This provides fast, accurate answers for known questions.

Stage 2 - FAISS Vector Search (Fallback)

If no Q/A match exceeds the threshold, the system falls back to searching the FAISS index built from all 15 scraped documentation pages.

The scraped documents were chunked using RecursiveCharacterTextSplitter (1000 characters, 100 character overlap) and converted to embeddings using the all-MiniLM-L6-v2 model. These were stored in a FAISS index and searched using L2 (Euclidean) distance.

Why RecursiveCharacterTextSplitter?

LangChain's RecursiveCharacterTextSplitter was chosen over manual word-based chunking because it splits text intelligently by trying paragraph breaks first, then line

breaks, then sentences, and finally words. It only goes smaller if the chunk is still too large. This preserves natural boundaries and avoids cutting text mid-sentence, which would reduce retrieval quality. A chunk size of 1000 characters was chosen because the LangChain documentation contains detailed technical explanations that require enough surrounding context to remain meaningful. A 100-character overlap ensures concepts that span chunk boundaries are not lost.

Why `llama-3.3-70b-versatile`?

Groq's `llama-3.3-70b-versatile` was selected as the LLM for answer generation for several reasons:

- Free API access — Groq provides free API usage, making it ideal for a student project
- Speed — Groq's LPU inference hardware runs Llama models significantly faster than standard GPU inference
- 70B parameter size — The 70-billion-parameter model provides much stronger reasoning and language generation compared to smaller variants like 8B or 13B
- Versatile — The "versatile" variant is optimized for general-purpose chat and question-answering tasks, which aligns well with this chatbot's use case
- Availability — The original `llama3-8b-8192` model was deprecated during development, and `llama-3.3-70b-versatile` became Groq's recommended replacement with improved capabilities

FAISS (Facebook AI Similarity Search) was chosen deliberately over higher-level alternatives such as ChromaDB or Pinecone for educational purposes. Using FAISS required manually managing the vector index, metadata storage (`metadata.pkl`), and the retrieval pipeline, which exposed the underlying mechanics of vector similarity search instead of abstracting them away.

FAISS index type — `IndexFlatL2`

`IndexFlatL2` was selected after evaluating the trade-offs between speed and accuracy:

Index Type	Method	Speed	Accuracy	Best For

IndexFlatL2	Brute force — compares every vector	Slowest	100% exact	Small datasets ✅ our case
IndexIVFFlat	Searches relevant clusters only	Fast	~95%	Medium datasets
IndexHNSW	Graph-based hierarchical search	Very fast	~98%	Large production datasets

With only around 300 chunks generated from 15 documentation pages, IndexFlatL2 was the most appropriate choice because it provides exact retrieval accuracy with no meaningful performance penalty at this scale.

FAISS returns the top three most similar chunks along with their source URLs stored separately in `metadata.pkl`. These retrieved chunks are then combined and passed as context to the LLM for final answer generation.

Off-Topic Guard

After the FAISS search, the best match distance is checked before calling the LLM. If the closest document has an L2 distance greater than `1.2`, the query is considered off-topic and a fixed response is returned immediately — the LLM is never called.

For `all-MiniLM-L6-v2` embeddings, L2 distance ranges from `0` (identical) to `~2` (opposite meaning). A distance above `1.2` means the query is not meaningfully close to anything in the knowledge base. This prevents the LLM from hallucinating answers to questions like "What is my name?" where retrieved context is irrelevant but the model answers from general knowledge anyway.

Why Hybrid?

Memory

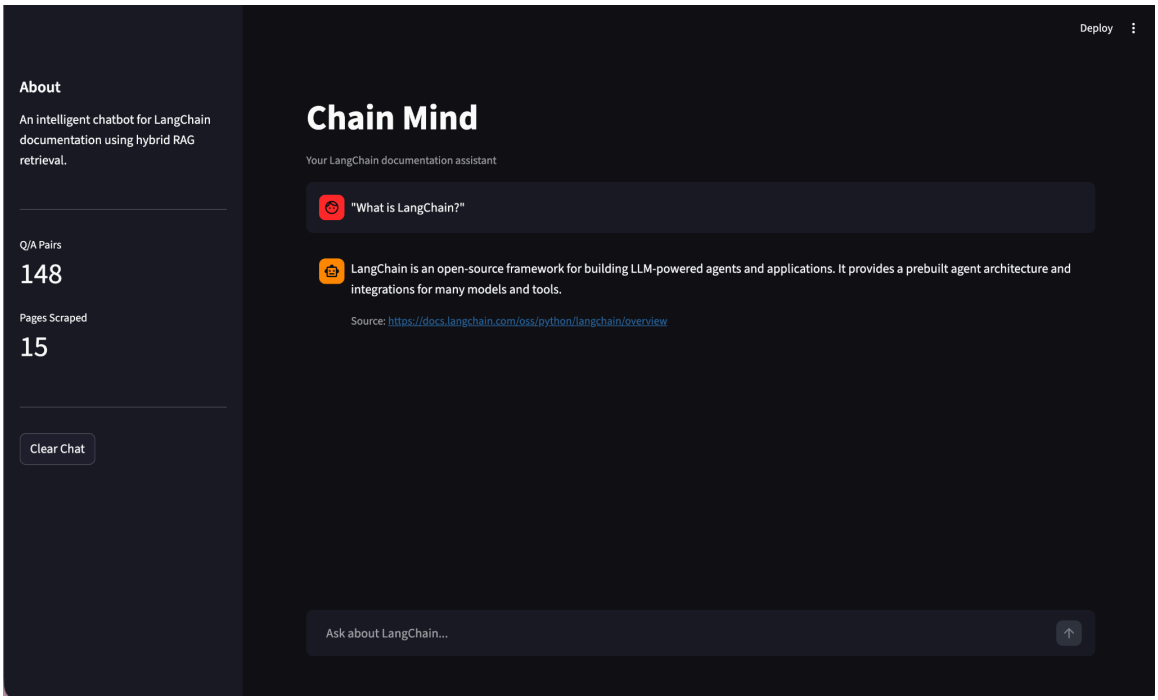
Approach	Strength	Weakness
Q/A only	Fast, precise for known questions	Fails for novel questions
Vector only	Handles any question	Answers may be less precise
Hybrid	Best of both worlds	Slightly more complex

Chat history is maintained by passing previous messages to the Groq API in each request. Streamlit's st.session_state stores messages in the browser session, which are passed as chat_history to the ask() function on every query.

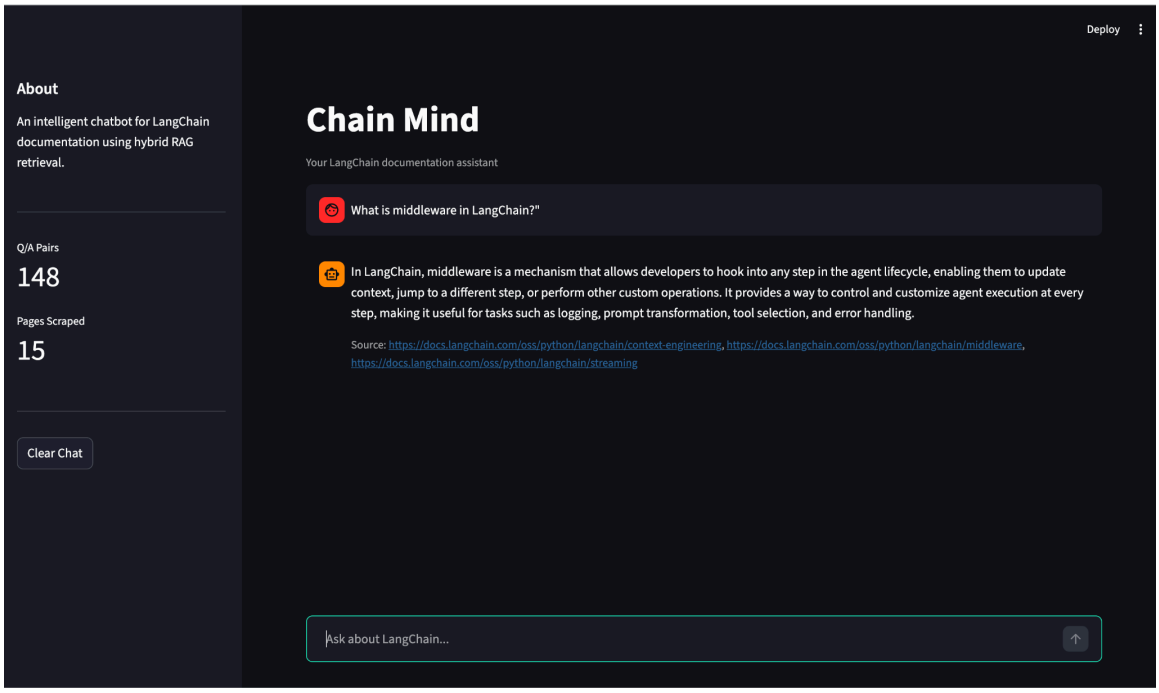
---

6. Screenshots

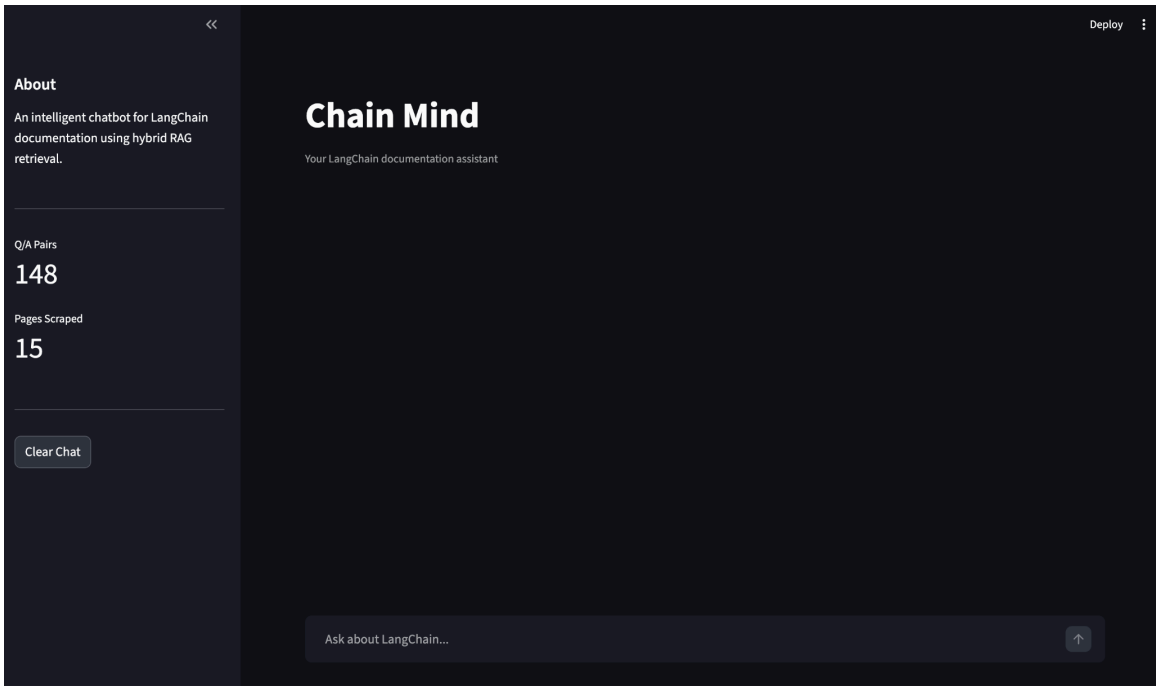
1. The full chat interface with a Q/A match response:



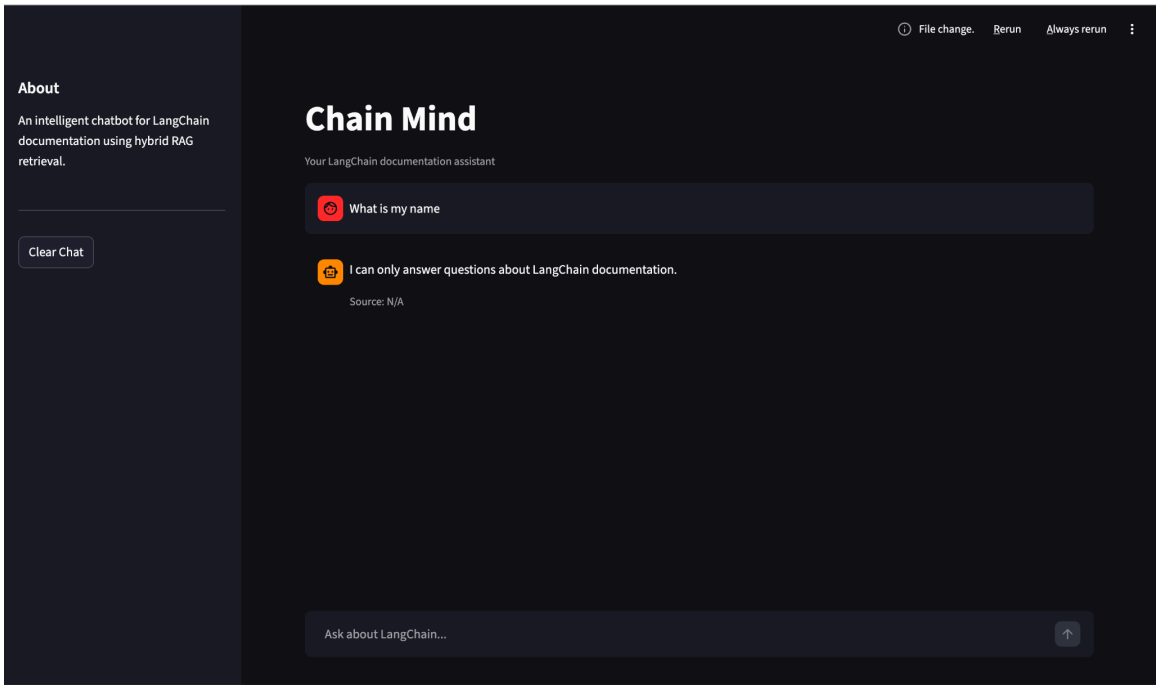
2. A vector search fallback response with source citation:



3. The sidebar showing Q/A pairs and pages scraped stats



4. Asking irrelevant questions:



---

7. Limitations & Future Improvements

Current Limitations

Q/A Coverage

- The curated Q/A dataset only covers the LangChain Overview page. Questions about other topics (agents, memory, RAG) fall back to vector search, which may produce less precise answers.
- Single-Page Q/A Dataset
- Expanding the Q/A dataset to cover all 15 scraped pages would significantly improve answer quality for the first retrieval stage.
- The current system returns the top FAISS result without re-ranking. A cross-encoder re-ranker could improve result relevance.

Static Knowledge Base

The scraped data is a snapshot in time. As LangChain documentation updates, the knowledge base becomes outdated. There is no automated refresh mechanism.

FAISS IndexFlatL2

The current FAISS index uses brute-force search (IndexFlatL2), which is accurate but slow for very large datasets. For production use with millions of documents, IndexIVFFlat or IndexHNSW would be more efficient.

Future Improvements

1. Expand Q/A dataset - Generate or curate Q/A pairs for all 15 scraped pages to improve first-stage retrieval coverage

2. Automated scraping refresh - Schedule periodic re-scraping to keep the knowledge base up to date

3. Re-ranking layer - Add a cross-encoder model to re-rank FAISS results before sending to the LLM

4. Switch to ChromaDB - For easier metadata management and a more production-ready vector store

5. FastAPI backend - Separate the backend into a REST API to support async requests and multiple concurrent users

6. Authentication - Add user login to support personalized chat history across sessions

7. Feedback mechanism - Allow users to rate answers, which could be used to improve the Q/A dataset over time

8. Streaming responses - Use Groq's streaming API to display answers token by token for a better user experience

---

Learnings

Building Chain Mind introduced me to a set of concepts I had no prior experience with, specifically around how meaning can be stored mathematically, searched efficiently, and used to ground an LLM's answers.

Embeddings turn meaning into math

The most conceptually new idea in this project was that text can be converted into a list of numbers (a vector) in a way that preserves meaning — similar sentences end up close together in that number space. Understanding how `sentence-transformers` produces 384-dimensional vectors, and that the position of a vector encodes semantic meaning, was a new mental model entirely.

Cosine similarity measures direction, not magnitude

Implementing the Q/A search manually using NumPy taught me why cosine similarity is preferred over plain dot product or Euclidean distance: it measures the angle between two vectors, not their size. A short question and a long passage about the same topic can point in the same direction even though one vector is much "larger." Pre-computing norms outside the loop was also a concrete example of why algorithmic efficiency matters even at small scale.

FAISS and how vector databases actually work

Before this project, vector databases were a black box. Building the FAISS pipeline manually — creating the index, storing embeddings, writing metadata to a pickle file, searching by L2 distance, and mapping result indices back to source text — exposed the mechanics underneath. Comparing `IndexFlatL2` (exact brute force) against `IndexIVFFlat` (cluster-based approximate) and `IndexHNSW` (graph-based) made the accuracy/speed tradeoff concrete rather than theoretical.

RAG is prompt engineering plus retrieval

The core insight of RAG is simple: give the LLM the relevant information before asking the question. Building the chatbot showed how the prompt structure — system message, retrieved context, user question — shapes the quality and reliability of the answer. Seeing the difference between answers with good context versus poor context made this concrete.